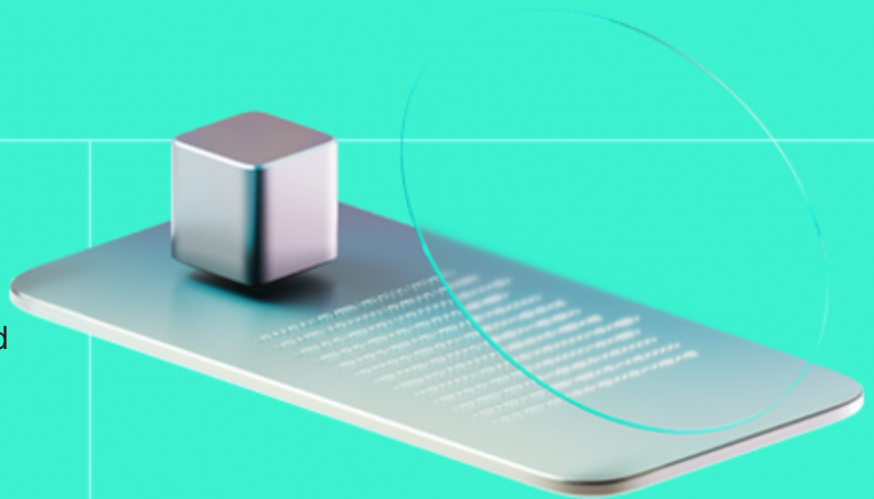




Smart Contract Code Review And Security Analysis Report

Customer: NEXPACE Limited

Date: 26/08/2025



We express our gratitude to the NEXPACE Limited team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Nexon introduced some of the biggest innovations in industry including the world's first graphic massively multiplayer online role-playing game (MMORPG) and the first free-to-play game.

Document

Name	Smart Contract Code Review and Security Analysis Report for NEXPACE Limited
Audited By	Seher Saylik, Khrystyna Tkachuk
Approved By	Ataberk Yavuzer
Website	https://msu.io/
Changelog	14/08/2025 - Preliminary Report 26/08/2025 - Final Report
Platform	HENESYS Chain, Avalanche C-Chain
Language	Solidity
Tags	Multisig Wallet, Reward Distributor
Methodology	https://hacken.io/cc/sc_methodology

Review Scope

Repository #1	Non Retrievable Multisig: https://github.com/NEXPACE-Limited/multisig-contract
Repository #2	Contributor Reward Distributor: https://github.com/NEXPACE-Limited/nexpaace-contracts
Commit #1	MultisigNonRetrievable - e703d275acc601be04d71e59897018f832e3f0b1
Commit #2	Contributor Reward Distributor - 6e8f0b1973a99b53b50b45dd5975e6800fe19a38
Retest #1	MultisigNonRetrievable - 27e905f0cb1f7add4d77ab1d87a1d9dba6ca7f57

Review Scope

Retest #2

Contributor Reward Distributor -

6e8f0b1973a99b53b50b45dd5975e6800fe19a38

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

7	0	6	1
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	1

Vulnerability	Severity
F-2025-12062 - Threshold Can Be Set to Owner Count or Zero, Leading to Governance Fragility and Signature Bypass	Low
F-2025-12053 - Floating Pragma	Info
F-2025-12054 - Lack of Event Emitting for Key State Changes	Info
F-2025-12056 - Missing Zero Address Check in Emergency Withdraw	Info
F-2025-12125 - Custom Errors Can Be Used for Gas Efficiency	Info
F-2025-12128 - Public Function not Used Internally can be Marked External	Info
F-2025-12162 - Gas-Inefficient Loop Increments in validateSignatures	Info

Documentation quality

- Functional requirements are not provided.
- A technical description is not provided.

Code quality

- Several inefficient gas usages were found.
- The development environment is configured.

Test coverage

Code coverage for `MultisigNonRetrievable` is **100%**, for `ContributionRewardDistributor` **100%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Interactions by several users are tested thoroughly.

Table of Contents

System Overview	7
Privileged Roles	7
Potential Risks	9
Findings	10
Vulnerability Details	10
Disclaimers	24
Appendix 1. Definitions	25
Severities	25
Potential Risks	25
Appendix 2. Scope	26
Appendix 3. Additional Valuables	27

System Overview

Nexpace is a multisignature transaction execution system with strict role separation between owners and executors, combined with auxiliary reward distribution logic with the following contracts:

OwnerManager — manages the list of owners who can sign transactions and the threshold of required confirmations.

ExecutorManager — maintains the list of executors who can generate, execute, or cancel transactions.

MultisigNonRetrievable — implements EIP-712-based multisignature transaction flow with explicit transaction status tracking:

- **Roles:**
 - Owners — sign transactions.
 - Executors — create, execute, and cancel transactions (can also be owners if explicitly set at deployment, but role addition is restricted thereafter).
- Each transaction is uniquely identified by `(requester, to, value, gas, salt, data)`.
- Certain sensitive function selectors are permanently prohibited from being called via this multisig:
 - `transferOwnership(address)`
 - `retrieve(address,address,uint256,string)`
 - `retrieve(address,address,uint256,uint256,string)`

TokenHolder — implements `ERC721Holder` and `ERC1155Holder` to allow the multisig to receive NFTs and ERC-1155 tokens, along with native ETH via `receive()`.

ContributionRewardDistributor — a separate reward distribution contract unrelated to the multisig, used to emit weekly contribution rewards based on a decaying exponential model until the year 2100.

- Distributes rewards in NXP to a pre-set contributionRewardWallet.
- Uses `taylorNegExp` approximation for exponential decay.
- Tracks weeks already distributed via `isDistributed`.
- Supports emergency withdrawals by the owner.

Privileged roles

- **Owners** (in `OwnerManager`)
Can update the owner set, change the threshold, and indirectly control which signers are valid.
Through majority signing power, they can authorize arbitrary transactions from the multisig.
- **Executors** (in `ExecutorManager`)
Can generate and submit transactions for execution or cancellation, but still requires owner signatures to pass threshold checks.

- **Owner of** `ContributionRewardDistributor`

Can update the `contributionRewardWallet` at any time and withdraw any ETH held in the contract via `emergencyWithdraw`.

Potential Risks

- The `ContributionRewardDistributor` contract allows the owner to withdraw all accumulated rewards at any time via `emergencyWithdraw`, introducing a centralization risk and undermining the trust assumptions of a scheduled distribution system.
- The `ContributionRewardDistributor` contract returns zero rewards for very large week numbers (e.g., $n > 8000$) due to underflow in the exponential decay calculation, potentially breaking the reward distribution schedule for distant future dates.
- In the current design of `MultisigNonRetrievable`, only the person who generates a transaction can execute it once the required number of signatures has been collected. If that person becomes unavailable, the transaction cannot be executed by others, potentially leaving the associated funds/operations permanently inaccessible.
- In `generateTransaction()` of `MultisigNonRetrievable`, the selector filter only inspects the top-level calldata, so `transferOwnership()` or `retrieve()` in target contracts can still be invoked indirectly via internal calls, wrapper contracts, or upgraded implementations.

Findings

Vulnerability Details

[F-2025-12062](#) - Threshold Can Be Set to Owner Count or Zero, Leading to Governance Fragility and Signature Bypass - Low

Description:

The `OwnerManager` contract allows the multisig threshold to be set equal to the total number of owners and even to zero under certain conditions.

```
require(  
    threshold_ <= owners_.length,  
    "OwnerManager/invalidThreshold: threshold must be less than or equal to l  
ength of owners"  
);
```

No lower-bound check (`threshold_ >= 1`) is enforced here, allowing `threshold_ = 0` at deployment.

During runtime:

```
require(  
    _owners.length() >= newThreshold,  
    "OwnerManager/invalidThreshold: threshold must be less than or equal to le  
ngth of owners"  
);
```

This enforces only the upper bound but still permits the threshold to equal the total number of owners.

• Zero Threshold → Signature Bypass

- When `threshold()` returns `0`, the signature validation logic in `validateSignatures` will always return true. As a result, any executor or owner can execute a generated transaction without providing any valid signatures, completely bypassing multisig protection.

• Threshold Equal to Owner Count → Governance Fragility

- If even one owner becomes unresponsive, loses access, or becomes malicious, the multisig becomes permanently frozen.
- This creates an **all-or-nothing trust model**, which is typically discouraged in decentralized or multi-actor protocols.

- This reduces fault tolerance and increases operational friction in real-world scenarios where temporary owner unavailability is common.

Assets:

- MultisigNonRetrievable.sol [<https://github.com/NEXPACE-Limited/multisig-contract/blob/feature/add-nonretrievable-multisig/contracts/MultisigNonRetrievable.sol>]

Status:

Mitigated

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations**Remediation:**

- Consider enforcing that the threshold is **strictly less than** the number of owners.
- Enforce a lower bound of 1 on threshold during deployment and threshold changes.

Resolution:

A validation check ensuring that the threshold is always greater than 1 has been implemented in the `constructor()` function.

The Nexpace team clarified that the number of owners will never equal the threshold, as this condition will be enforced off-chain.

Therefore, this part of the issue is considered mitigated and out of this audit's scope.

(Revised commit: `27e905f`)

F-2025-12053 - Floating Pragma - Info

Description:

In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.20`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations `^0.8.17` in `MultisigNonRetrievable.sol` contract and its dependencies (`OwnerManager`, `ExecutorManager`, `TokenHolder`). This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- MultisigNonRetrievable.sol [<https://github.com/NEXPACE-Limited/multisig-contract/blob/feature/add-nonretrievable-multisig/contracts/MultisigNonRetrievable.sol>]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler

version, preventing unexpected changes in behavior due to compiler updates.

Resolution:

The risk of the given finding is accepted and no further changes are applied.

F-2025-12054 - Lack of Event Emitting for Key State Changes - Info

Description:

The `ContributionRewardDistributor` contract is responsible for distributing weekly contribution rewards by transferring them to the `contributionRewardWallet` address within the `distribute()` function. The `setContributionRewardWallet()` function allows the contract owner to update the `contributionRewardWallet` address.

```
function setContributionRewardWallet(
    address newContributionRewardWallet
) external onlyOwner validAddress(newContributionRewardWallet) {
    contributionRewardWallet = newContributionRewardWallet;
}
```

However, these key state changes do not emit any events. The absence of an event makes it difficult to track admin-specific actions off-chain, which reduces transparency and complicates monitoring and auditing.

Additionally, the contract owner can withdraw funds at any time via the `emergencyWithdraw()` function. However, this action does not emit any event upon execution. As a result, the fund transfer is neither publicly notified nor easily monitored and indexed by off-chain services. The absence of an event significantly reduces transparency and makes it harder for auditors, monitoring tools, and users to track fund movements or detect unauthorized withdrawals.

```
function emergencyWithdraw(address to, uint256 amount) external onlyOwner {
    (bool success, ) = to.call{ value: amount }("");
    require(success, "ContributionRewardDistributor/transferFailed: NXP transfer failed");
}
```

It is recommended to emit events for this operation to ensure proper visibility, support off-chain indexing, and facilitate debugging, especially for critical administrative and user-facing fund transfers.

Assets:

- ContributorRewardDistributor.sol [<https://github.com/NEXPACE-Limited/nexpace-contracts/blob/feature/add-contribution-distributor/contracts/ContributionRewardDistributor/ContributionRewardDistributor.sol>]

Status: Accepted

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Consider emitting events within the `setContributionRewardWallet()` and `constructor()` functions to improve transparency and enable reliable off-chain tracking of key state changes of `contributionRewardWallet`.

```
event ContributionRewardWalletSet(address indexed newWallet);
```

Additionally, consider adding event emission upon `emergencyWithdraw()` function execution:

```
event EmergencyWithdrawal(address indexed to, uint256 amount);
```

Emitting these events is a best practice as it enables external observers, such as monitoring tools, analytics dashboards, and indexers, to accurately reflect contract activity, facilitates debugging.

Resolution: The risk of the given finding is accepted and no further changes are applied.

[F-2025-12056](#) - Missing Zero Address Check in Emergency Withdraw - Info

Description: The `emergencyWithdraw()` function allows the contract owner to transfer ETH from the contract to an arbitrary address. However, the function does not include a check to ensure that the `to` address is not the zero address (`address(0)`):

```
(bool success, ) = to.call{ value: amount }("");
```

If `to` is set to `address(0)`, the transfer will succeed and the ETH will be irrecoverably burned. This could happen due to user error, misconfiguration, or misuse.

Impact:

Permanent loss of funds if the zero address is provided as the withdrawal target.

Weakens contract safety and may result in accidental fund destruction.

Assets:

- ContributorRewardDistributor.sol [<https://github.com/NEXPACE-Limited/nexpace-contracts/blob/feature/add-contribution-distributor/contracts/ContributionRewardDistributor/ContributionRewardDistributor.sol>]

Status:

Accepted

Classification

Impact Rate: 3/5
Likelihood Rate: 1/5
Exploitability: Dependent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Add a validation check before the transfer to ensure the recipient is not the zero address:


```
require(to != address(0), "ContributionRewardDistributor/invalidRecipient: zero address");
```

Resolution:

The risk of the given finding is accepted and no further changes are applied.

F-2025-12125 - Custom Errors Can Be Used for Gas Efficiency - Info

Description:

Starting from Solidity version 0.8.4, the language introduced a feature known as "custom errors". These custom errors provide a way to define more descriptive and semantically meaningful error conditions without relying on string messages. Prior to this version, the `require` statement with string error messages was commonly used to handle specific conditions or validations. However, each unique string used as a revert reason consumes gas, increasing transaction costs.

Within the `MultisigNonRetrievable` and `ContributionRewardDistributor` contract codebases, there are several `require` statements with significant revert messages. These revert messages, while useful for debugging, can be gas-inefficient due to the consumption of gas for each unique string used:

```
require(
    _txStatus[txId] == Status.UNDEFINED,
    "Multisig/invalidRequest: transaction has already been generated, cancelled
    , or executed"
);
require(bytes4(data) != 0x687fc0b5, "Multisig/nonRetrievable: retrieve functi
on is prohibited");
require(isOwner(signer), "Multisig/invalidSignature: signer is not owner");
require(success, "ContributionRewardDistributor/transferFailed: NXP transfer
failed");
require(n != 0, "ContributionRewardDistributor/invalidWeek: week cannot be ze
ro");
...
```

Custom errors, on the other hand, are identified by their name and the types of their parameters only, and they do not have the overhead of string storage. This means that, when using custom errors instead of `require` statements with string messages, the gas consumption can be significantly reduced, leading to more gas-efficient contracts.

Assets:

- `MultisigNonRetrievable.sol` [<https://github.com/NEXPACE-Limited/multisig-contract/blob/feature/add-nonretrievable-multisig/contracts/MultisigNonRetrievable.sol>]
- `ContributorRewardDistributor.sol` [<https://github.com/NEXPACE-Limited/nexpace-contracts/blob/feature/add-contribution->

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: It is recommended to use custom errors instead of revert strings to reduce gas costs, especially during contract deployment. Custom errors can be defined using the `error` keyword and can include dynamic information, for example:

```
error InvalidSigner(address signer);
error InvalidSignature();
...
revert InvalidSigner(signer)
revert InvalidSignature();
```

Resolution:

The risk of the given finding is accepted and no further changes are applied.

[F-2025-12128](#) - Public Function not Used Internally can be Marked External - Info

Description: The `transferAmount()` function in the `ContributionRewardDistributor` contract is currently marked as `public`, but it is not invoked internally within the contract itself. Since this function is only intended to be called externally, it should be marked as `external` instead of `public`.

In Solidity, functions that are only meant to be invoked by external actors, such as external contracts or users, should be designated as `external` rather than `public`. While both public and external functions are accessible from outside the contract, there are key differences.

A public functions can be called both from other functions inside the same contract and from outside transactions, while an external function can only be called from outside the contract. Specifically, external functions are more gas-efficient when called externally, as they do not require the extra internal checks that public functions do. Marking a function as `public` instead of `external` introduces unnecessary gas costs.

Assets:

- ContributorRewardDistributor.sol [<https://github.com/NEXPACE-Limited/nexpace-contracts/blob/feature/add-contribution-distributor/contracts/ContributionRewardDistributor/ContributionRewardDistributor.sol>]

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider changing `transferAmount()` visibility to `external` instead of `public` to enhance gas efficiency and reduce unnecessary exposure. This change ensures that the function is only callable from external

sources, reducing internal contract complexity and improving security.

Resolution:

The risk of the given finding is accepted and no further changes are applied.

[F-2025-12162](#) - Gas-Inefficient Loop Increments in validateSignatures - Info

Description: In `validateSignatures()`, the loop uses the default `i++` increment inside a standard `for` statement:

```
for (uint256 i = 0; i < signatures.length; i++) {  
    // ...  
    confirmCount += 1;  
}
```

This incrementation is checked for overflow on every iteration, costing extra gas. Since `i` is strictly bounded by `signatures.length` (an external calldata array) and cannot approach `type(uint256).max`, overflow is impossible in practice.

Changing the increment to an **unchecked** form inside the loop body avoids redundant overflow checks and slightly reduces gas costs:

```
for (uint256 i = 0; i < signatures.length; ) {  
    // ...  
    confirmCount += 1;  
    unchecked { i++; }  
}
```

Assets:

- MultisigNonRetrievable.sol [<https://github.com/NEXPACE-Limited/multisig-contract/blob/feature/add-nonretrievable-multisig/contracts/MultisigNonRetrievable.sol>]

Status: Accepted

Classification

Impact Rate: 1/5
Likelihood Rate: 5/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

- Remediation:** Use unchecked increments for both `i` and optionally `confirmCount` to reduce runtime gas cost, ensuring loop boundaries are safe.
- Resolution:** The risk of the given finding is accepted and no further changes are applied.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

A reference on how risk scoring is done is available through the repository in our GitHub organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository #1	Non Retrievable Multisig: https://github.com/NEXPACE-Limited/multisig-contract
Repository #2	Contributor Reward Distributor: https://github.com/NEXPACE-Limited/nexpace-contracts
Commit #1	MultisigNonRetrievable - e703d275acc601be04d71e59897018f832e3f0b1
Commit #2	Contributor Reward Distributor - 6e8f0b1973a99b53b50b45dd5975e6800fe19a38
Retest #1	MultisigNonRetrievable - 27e905f0cb1f7add4d77ab1d87a1d9dba6ca7f57
Retest #2	Contributor Reward Distributor - 6e8f0b1973a99b53b50b45dd5975e6800fe19a38
Whitepaper	N/A
Requirements	N/A
Technical Requirements	N/A

Asset	Type
ContributorRewardDistributor.sol [https://github.com/NEXPACE-Limited/nexpace-contracts/blob/feature/add-contribution-distributor/contracts/ContributionRewardDistributor/ContributionRewardDistributor.sol]	Smart Contract
ExecutorManager.sol [https://github.com/NEXPACE-Limited/multisig-contract/blob/feature/add-nonretrievable-multisig/contracts]	Smart Contract
MultisigNonRetrievable.sol [https://github.com/NEXPACE-Limited/multisig-contract/blob/feature/add-nonretrievable-multisig/contracts/MultisigNonRetrievable.sol]	Smart Contract
OwnerManager.sol [https://github.com/NEXPACE-Limited/multisig-contract/blob/feature/add-nonretrievable-multisig/contracts]	Smart Contract
TokenHolder.sol [https://github.com/NEXPACE-Limited/multisig-contract/blob/feature/add-nonretrievable-multisig/contracts]	Smart Contract

Appendix 3. Additional Valuables

Verification of System Invariants

During the audit of NEXPACE Limited, Hacken followed its methodology by performing fuzz-testing on the project's main functions. [Foundry](#), a tool used for fuzz-testing, was employed to check how the protocol behaves under various inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific Echidna fuzzing suite was prepared for this task, and throughout the assessment, 10 invariants were tested over 10,000 runs. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

Invariant	Test Result	Run Count
<code>MultisigNonRetrievable</code> : For each <code>txId</code> ever seen: <code>status ∈ {GENERATED, CANCELLED, EXECUTED}</code> and it never goes back to <code>UNDEFINED</code> or <code>GENERATED</code> after reaching a terminal state	Passed	10k
<code>MultisigNonRetrievable</code> : threshold bounds should always be " $1 \leq \text{threshold}() < \text{ownersCount}$ ".	Failed	10k
<code>ContributionRewardDistributor</code> : all weekly rewards for 10 years should align with the provided 10-year reward list.	Passed	10k
<code>ContributionRewardDistributor</code> : reward amount should never overflow and return as zero in <code>transferAmount()</code> .	Failed	10k
<code>ContributionRewardDistributor</code> : non-owner should not be able to pause.	Passed	10k
<code>MultisigNonRetrievable</code> : All generated <code>txId</code> must be unique	Passed	10k
<code>MultisigNonRetrievable</code> : Only original requester can cancel/execute its <code>txId</code>	Passed	10k
<code>MultisigNonRetrievable</code> : Only valid signature can be used for signing execute operation	Passed	10k
<code>MultisigNonRetrievable</code> : Owner and executor can't be the same	Passed	10k
<code>MultisigNonRetrievable</code> : Owners can't duplicate	Passed	10k

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.